Université Paris Cité, Campus Saint-Germain-des-Près
UFR de Mathématiques et Informatique
45, rue des Saints-Pères 75006 Paris

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Algorithmique et Programmation 2

Contrôle terminal du 16 mai 2025 — durée: 1h30

Correction

Tout document interdit. Les calculatrices et les téléphones portables, même prévus à titre d'horloge, sont également interdits.

MERCI DE BIEN INDIQUER VOTRE GROUPE DE TD SUR VOTRE COPIE

Tous les exercices sont indépendants. On précisera bien la signature (le type des entrées/sorties suffit) pout chaque fonction (Python ou pseudo-code) proposée. Sur la note globale, 1 point sera attribué en fonction de la proportion de signatures correctes.

Exercice 1. Pour chaque ligne (indépendante) d'instructions ci-dessous, donner le contenu de la variable `x` après exécution de la ligne considérée si l'exécution de la ligne ne provoque pas d'erreur, ou expliquer la nature de l'erreur dans le cas contraire. **Expliquer votre réponse dans les deux cas (une réponse correcte sans explication ne sera pas comptabilisée).**

```
1  x = (1,2,3); x.append(4)
2  x = [1,2,3].append(4)
3  x = {1:{1,2,3}, 3:{1,2,3}}; x[1].remove(3)
4  x = ({1,2}|{1,3}) - ({1,3}&{1,2})
5  L = [i**2 for i in range(-6,3,2)]; x = len(set(L))
6  L = set(range(10)); x = [{10-2*i,i*3} <= L for i in range(5)]
7  A = frozenset({1}); S = frozenset(A) | frozenset([A]); x = len(S)
8  x = {1:2,2:3}.get(0,1) - {1:2,2:3}.get(1,0)
9  x = "abc"; x[0],x[2] = x[2],x[0]
10 s = "123"; x = s[s[1]]
```

(1) Ligne 1: provoque une erreur, car le type `tuple` n'a pas de méthode `append` (conséquence du fait que le type `tuple` est immutable).

(2) Ligne 2: `x = None`, car la méthode `append`, appliquée à la liste `[1,2,3]`, ne retourne rien (cette méthode modifie la liste `[1,2,3]` quelque part en mémoire, mais c'est sans rapport avec `x`).

(3) Ligne 3: `x = {1: {1, 2}, 3: {1, 2, 3}}`, car la deuxième instruction `x[1].remove(3)` supprime l'élément 3 de la valeur associée à la clé 1 dans le dictionnaire `x` initial.

(4) Ligne 4: `x = {2, 3}`, car `x` est défini comme l'ensemble $(\{1,2\}\cup\{1,3\})\setminus(\{1,3\}\cap\{1,2\}) = \{1,2,3\}\setminus\{1\} = \{2,3\}$).

(5) Ligne 5: `x = 4`. `range(-6,3,2)` correspond aux valeurs -6, -4, -2, 0, 2, donc `L = [36, 16, 4, 0, 4]`, `set(L) = {36, 16, 4, 0}` et donc `len(set(L)) = 4`.

(6) Ligne 6: `x = [False, True, True, True, False]`. On commence par construire `L`, qui est l'ensemble des entiers de 0 à 9 inclus. Puis `x` est une liste de booléens obtenue en testant, pour chaque valeur de i entre 0 et 4 inclus, si l'ensemble $\{10-2i, 3i\}$ est inclus dans `L`.
Pour $i = 0$, $\{10-2i, 3i\} = \{10, 0\} \not\subset \{0, 1, \ldots, 9\} \longrightarrow$ `False`

Pour $i = 1$, $\{10 - 2i, 3i\} = \{8, 3\} \subset \{0, 1, \ldots, 9\} \longrightarrow$ `True`
Pour $i = 2$, $\{10 - 2i, 3i\} = \{6\} \subset \{0, 1, \ldots, 9\} \longrightarrow$ `True`
Pour $i = 3$, $\{10 - 2i, 3i\} = \{4, 9\} \subset \{0, 1, \ldots, 9\} \longrightarrow$ `True`
Pour $i = 4$, $\{10 - 2i, 3i\} = \{2, 12\} \not\subset \{0, 1, \ldots, 9\} \longrightarrow$ `False`

(7) Ligne 7: `x=2`. `A` est l'ensemble (de type `frozenset`) $\{1\}$, `frozenset(A)` aussi (la conversion `frozenset` en `frozenset` ne change rien bien sûr), et `frozenset([A])` est un `frozenset` dont l'unique élément est le `frozenset` $\{1\}$, car c'est la conversion en `frozenset` d'une liste formée du `frozenset` $\{1\}$. Au final on a $S = \{1, \{1\}\}$ (tous les ensembles considérés étant de type `frozenset`), donc `len(S)=2`.

(8) Ligne 8: `x = -1`, car `{1:2,2:3}.get(0,1)` retourne 1 (la clé 0 n'est pas dans le dictionnaire `{1:2,2:3}`, donc c'est la valeur par défaut, le 2e argument de `get`, qui est retournée) et `{1:2,2:3}.get(1,0)` retourne 2, la valeur associée à la clé 1 dans le dictionnaire `{1:2,2:3}`.

(9) Ligne 9: erreur, car `x` est de type `str`, qui est immutable, donc on ne peut pas affecter `x[0]` (ni `x[2]` bien sûr).

(10) Ligne 10: erreur, car `s[1]` retourne la chaîne de caractère `"2"`, qui n'est pas un index valide pour `s` car de type `str` et non `int`.

Exercice 2. Pour chaque fonction ci-dessous, proposez un algorithme **sous forme de pseudo-code** (les codes Python ne seront pas acceptés ici).

(1) La fonction récursive f_1 prend en entrée la racine d'un arbre, et retourne le nombre de nœuds de l'arbre ayant au moins une feuille parmi leurs enfants.

Il faut calculer à la fois la contribution du nœud courant `N` (0 ou 1 selon la valeur de `compte`, le nombre d'enfants de `N` qui sont des feuilles) et la somme des contributions des enfants de `N` (`total`, la somme des `f1(E)` pour `E` enfant de `N`). On peut faire deux boucles séparées, ou faire comme ci-dessous le calcul en parallèle dans la même boucle.

```
fonction f1(N)
    // noeud -> entier
    compte <- 0 // nombre de feuilles parmi les enfants de N
    total <- 0  // contributions à f1(N) venant des enfants de N
    pour tout enfant E de N
        total <- total + f1(E)
        si le nombre d'enfants de E est égal à 0
            compte <- compte + 1
    si compte >= 1
        total <- total + 1
    retourner total
```

(2) La fonction récursive f_2 prend en entrée la racine d'un arbre dont les nœuds sont étiquetés par des nombres positifs, et retourne le score maximal parmi les nœuds de l'arbre, où le score d'un nœud est défini par la somme des valeurs des étiquettes de ses enfants (par convention, 0 s'il n'a pas d'enfants).

Comme pour la question précédente, il faut à la fois calculer le score du nœud `N` (ici, la variable `score`), et le maximum des scores des enfants de `N` (ici, `m`). Ceci peut se faire en parallèle dans la même boucle sur les enfants `E` de `N`.

```
fonction f2(N)
    // noeud -> nombre
    score <- 0
    m <- 0
    pour tout enfant E de N
        m <- max(m, f2(E))
        score <- score + étiquette(E)
    retourner max(m, score)
```

(3) La fonction f_3 prend entrée une pile A formée de nombres entiers, et retourne une pile B formée des éléments strictement positifs de A, dans l'ordre inverse par rapport à l'ordre de départ.

Quand on recopie une pile dans une autre pile, l'ordre s'inverse naturellement, donc ici on a juste à enlever dans la recopie les éléments qui ne sont pas strictement positifs.

```
fonction f3(A)
    // pile -> pile
    B <- pile vide
    tant que A n'est pas vide
        dépiler A dans x
        si x > 0
            empiler x dans B
    retourner B
```

(4) La fonction f_4 prend en entrée un graphe G simple, non orienté, et retourne le booléen `vrai` si G admet au moins un cycle de longueur 3, et le booléen `faux` sinon.

Un cycle est un chemin qui relie un sommet à lui-même sans employer deux fois la même arrête. Comme le graphe est simple (aucune arête ne relie un sommet à lui-même), un cycle de longueur 3 est simplement un chemin fermé de longueur 3.

```
fonction f4(G)
    // graphe -> booléen
    pour tout sommet x de G
        pour tout voisin y de x
            pour tout voisin z de y
                si x est voisin de z
                    retourner vrai
    retourner faux
```

Exercice 3. Proposer une implémentation Python de chaque fonction ci-dessous.

(1) La fonction `g1` prend en entrée une liste d'entiers L non vide et retourne la liste des sommes partielles de L. Par exemple, `g1([2, 1, 0, 5])` doit retourner la liste `[2, 3, 3, 8]`.

On peut calculer la liste des cumuls en parcourant une seule fois la liste de départ, et en cumulant progressivement les valeurs:

```
def g1(L):
    "list -> list"
    cumul = 0
    S = []
    for x in L:
        cumul += x
        S.append(cumul)
    return S
```

La solution ci-dessous, bien que de complexité moins bonne ($O(n^2)$ si n est la taille de L, contre $O(n)$ pour la première solution), est aussi valable:

```
def g1(L):
    "list -> list"
    return [sum(L[:k+1]) for k in range(len(L))]
```

On acceptera sans pénalité les solutions qui modifient l'argument de la fonction, par exemple

```
def g1(L):
    "list -> list"
    for i in range(len(L)-1):
        L[i+1] += L[i]
    return L
```

(2) La fonction `g2` prend en entrée une chaîne de caractères `s` et retourne une liste de nombres `L` de même taille que `s`. Pour tout indice valide `i` de `s`, `L[i]` est le nombre d'occurrences de `s[i]` rencontrées dans `s` strictement avant l'indice `i`.
Par exemple, si `s="abacbbd"`, `g2(s)` doit retourner la liste `[0, 0, 1, 0, 1, 2, 0]`.
Cette fonction pourra utiliser un dictionnaire accessoire. Les solutions de complexité $O(n)$ (où n est la taille de s) se verront gratifiées d'un bonus de points.

Une première solution consiste à compter le nombre d'occurences de chaque caractère rencontré, par exemple:

```
def g2(s):
    " str -> list "
    return [s[:i].count(s[i]) for i in range(len(s))]
```

Mais cette solution a une complexité $O(n^2)$ (n étant la longueur de s), car la méthode `count` effectue un nouveau parcours de `s` à chaque fois.

Pour obtenir une solution de complexité $O(n)$, une méthode simple consiste à utiliser un dictionnaire accessoire pour garder le compte à chaque instant du nombre d'occurences de chaque caractère rencontré:

```
def g2(s):
    " str -> list "
    D = dict() # nombre d'occurences pour chaque élément de s, à
un instant donné
    L = []
    for x in s:
        n = D.get(x,0) # nombre d'occurences précédentes de x
        L.append(n)
        D[x] = n+1
    return L
```

(3) La fonction <u>récursive</u> `g3` qui prend en entrée un entier naturel n, et renvoie le terme u_n de la suite définie par $u_0 = 1$ et la relation de récurrence

$$\forall n \in \mathbb{N}, \quad u_{n+1} = \begin{cases} u_n^2 + 1 & \text{si } u_n \text{ est pair,} \\ u_n + n & \text{sinon.} \end{cases}$$

```
def g3(n):
    " int -> int "
    if n==0:
        return 1
    u = g3(n-1)
    if u%2==0:
        return u**2+1
    return u+n-1
```

(4) La fonction génératrice `g4` prend en entrée un ensemble non vide `S` de type `set`, et délivre tous les sous-ensembles de `S` obtenus en supprimant un seul élément de `S`. Par exemple, `g4({1,2,3})` doit délivrer les ensembles `{1,2}`, `{2,3}` et `{1,3}`.

```
def g4(S):
    " set -> generator "
    for x in S:
        yield S - {x}
```

(5) La fonction génératrice <u>récursive</u> `g5` prend en entrée un nombre entier $n \geqslant 1$, et délivre toutes les listes L formées d'entiers strictement positifs <u>en ordre croissant</u> (au sens large), dont la somme des éléments vaut n. Par exemple, `g5(5)` doit délivrer les listes

`[1,1,1,1,1], [1,1,1,2], [1,2,2], [1,1,3], [2,3], [1,4], [5].`

On pourra utiliser la remarque suivante: pour toute liste `B` délivrée par `g5(n)` de la forme `B = A + [p]`, `A` fait partie des listes délivrées par `g5(n-p)`.

```
def g5(n):
    " int -> generator "
    if n==1:
        yield [1]
    else:
        for last in range(1,n):
            for L in g5(n-last):
                if L[-1]<=last:
                    yield L+[last]
        yield [n]
```

Exercice 4. On souhaite manipuler avec une classe Python des nombres décimaux, sans contrainte sur le nombre de chiffres. Pour cela, on choisit de représenter un tel nombre à l'aide de deux attributs de type `int`, notés x et p (avec $p \geqslant 0$), qui ensemble codent le nombre $x.10^{-p}$.

Par exemple, le nombre décimal `-435,0072` peut être codé par $x = -4350072$ et $p = 4$, mais aussi, de manière équivalente, par $x = -43500720$ et $p = 5$ (on ne cherche pas à normaliser la représentation dans cet exercice).

Écrire un code Python permettant de définir une telle classe `Decimal` avec les fonctionnalités suivantes :

(1) `Decimal(x,p)` crée une instance de la classe correspondant au nombre $x.10^{-p}$.

(2) si `a` et `b` sont deux instances de la classe `Decimal`, alors `a*b` retourne une nouvelle instance de la classe `Decimal`, correspondant au produit des deux nombres décimaux codés par `a` et `b`.

(3) Si `a` est une instance de la classe `Decimal`, alors `a.int()` retourne un nombre de type `int` égal à la partie entière de `a`.

(4) Si `a` est une instance de la classe `Decimal`, l'affichage de `a` (par exemple avec `print(a)`) est réalisé comme dans les exemples ci-dessous:

```
>>> print(Decimal(-4350072,4))
-435.0072
>>> print(Decimal(42,0))
42.
>>> print(Decimal(3,2))
0.03
```

```python
class Decimal():

    def __init__(self, x, p):
        self.x = x
        self.p = p

    def __mul__(P, Q):
        return Decimal(P.x*Q.x, P.p+Q.p)

    def __repr__(self):
        s = str(self.x)
        if len(s) < self.p + 1: # il faut compléter s par des zéros à gauche
            s = "0"*(self.p + 1 - len(s)) + s
        return s[:len(s)-self.p]+"."+s[len(s)-self.p:]

    def int(self):
        return self.x//10**self.p
```